

```

#include <iostream>
#include <vector>
#include <unordered_map>

using namespace std;

int Iterative_BS(const int* array, int high, int target, int& compare_I){
    int low = 0;

    //make it check mid and then check left or right
    while(high >= low){
        int mid = (high + low)/2;

        // checks right side
        if(array[mid] < target){
            compare_I++;
            low = mid + 1;
        }

        //checks left side
        else if(array[mid] > target){
            compare_I++;
            high = mid - 1;
        }

        //Answered
        else{
            compare_I++;
            return mid;
        }
        //add on to the number of comparisons
        compare_I++;
    }
    return -1;
}

int Recursive_BS(const int* array, int low, int high, int target, int& compare_R){
    //if we have no array or if array sizes make no sense
    if(low > high){
        return -1;
    }

    int mid = (low + high)/ 2;

    //checks left
    if( target < array[mid]){
        compare_R++;
        return Recursive_BS(array, low, mid - 1, target, compare_R);
    }
    //checks right
    else if( array[mid] < target){
        compare_R++;
        return Recursive_BS(array, mid + 1, high, target, compare_R);
    }
    //adds to the number of comparisons and return answer
    compare_R++;
    return mid;
}

////////////////////////////////////
////////////////////////////////////

int main(){
    int low = 0;

```

```
////////////////////////////////Test for recursive////////////////////////////////
cout << "Starting testing for Recurcsive Binary Search" << endl << endl;
```

[https://raw.githubusercontent.com/RBurcia/CS-210-/refs/heads/main/Assignment2/Iterative BS.cpp](https://raw.githubusercontent.com/RBurcia/CS-210-/refs/heads/main/Assignment2/Iterative%20BS.cpp)

```
// The time complexity of the iterative search was  $O(\log n)$ , and the space complexity was  $O(1)$ 
// since we keep
// the same memory of the array. The time complexity of the recursive search was  $O(\log n)$ , and the
// space complexity is also  $O(\log n)$  since for each recursion we create a new pointer to the same array
// but half of it. We keep doing this repetition until we find the target.

// Why we have the  $T(n) = T(n/2) + O(1)$  refer to the same thing as  $O(\log n)$  for the recursive
// search, is
// because one time doing a loop in a recursion can be referred to as the same thing as  $T(n/2) + O(1)$ ,
// where
//  $O(1)$  refers to the first comparison in the middle of the array and  $T(n/2)$  refers to us going into
// the recursive
// function again, but taking half of the array instead. When we have big arrays we exponentially get
// closer to the
// target by taking half of the array each time, but instead of being slow regardless how
// exponentially bigger
// our input is, it becomes fast regardless of how exponentially big our input size is.

    return 0;
}
```